

CHAPTER 5

IMPLEMENTATION AND TESTING

Implementation means building the project as planned, such as writing code, developing features, and integrating different modules. Testing means checking the developed project to find and fix errors. Together, they ensure the system works correctly and meets requirements.

5.1 IMPLEMENTATION APPROACHES

Implementation approaches are the strategies or methods used to introduce a new system or project into real use. Since every project either replaces or improves an existing system, the way it is implemented is very important to reduce risks, avoid failures, and make the system acceptable to users.

- **Direct Implementation:** Immediately replacing the old donation/management system (if any) with the new Digital Fundraising Portal. It is fast and simple but risky, as any failure can affect all users and fundraising campaigns.
- **Parallel Implementation:** Both the old method (manual donations or existing platform) and the new system run at the same time until the new one is proven reliable. This method is safe but requires more resources.
- **Phased Implementation:** The new system is introduced step by step, such as first enabling user registration and login, then campaign creation, then donation processing. This lowers risks and allows gradual improvements but takes more time.
- **Pilot Implementation:** The system is tested on a small group of fundraisers and donors first. If it works well, it is expanded to the full organization. This reduces risks but may not reveal large-scale performance issues.

5.2 CODING DETAILS AND CODE EFFICIENCY

5.2.1 CODING DETAILS

Coding details describe how the Digital Fundraising Portal was developed. The project uses React (frontend), Supabase/PostgreSQL (database), and Node.js APIs/Edge Functions for backend processing.

- Code is structured into modules (authentication, campaigns, donations, reports, admin panel).
- Variables and functions follow consistent naming conventions.
- Comments are added for better readability and debugging.
- Authentication and database access are implemented using Supabase libraries.
- Payment integration is modular for easy updates.
- This modular coding structure makes the system easy to understand, maintain, and extend.

5.2.2 CODE EFFICIENCY

- Code efficiency ensures the portal runs smoothly with limited resources.
- Optimized SQL queries improve database performance.
- Use of UUIDs for campaign and user IDs prevents duplication.
- Asynchronous operations (e.g., donations, payment confirmation) improve speed.
- Minimal API calls reduce server load.
- Reusable components in React reduce redundancy.
- Efficient coding ensures faster performance, reduced costs, and reliable fundraising operations.

5.3 TESTING APPROACH

Testing approach explains the overall plan to ensure the system works correctly. The Digital Fundraising Portal was tested using multiple approaches:

5.3.1 UNIT TESTING

- Each module (login, campaign creation, donation) tested individually.
- Example: Testing donation calculation logic without connecting to the full system.

5.3.2 INTEGRATION TESTING

- Modules combined and tested together.
- Example: Campaign creation linked with donation processing and reporting.

5.3.3 SYSTEM TESTING

- The whole portal tested as a complete system.
- Ensured campaigns, donations, authentication, and reporting all work as expected.
- Verified end-to-end workflows from user registration → donation → report generation.
- System testing ensured:
 - The whole portal functions properly.
 - Requirements are met.
 - Errors are identified and fixed.
 - Confidence before delivery is achieved.

5.3.4 USER ACCEPTANCE TESTING (UAT)

- Final testing done with real users (donors and fundraisers).
- Verified usability, reliability, and accuracy.
- Confirmed the portal solves the fundraising problem and is ready for deployment.

5.3.5 BLACK BOX TESTING

- Tested inputs and outputs without checking code.
- Example: Donor enters amount → System processes → Output is donation success message.

Techniques used:

- Equivalence Partitioning (e.g., valid/invalid amounts).
- Boundary Value Analysis (e.g., minimum/maximum donation amounts).
- Decision Table Testing (different payment scenarios).
- Error Guessing (invalid campaign ID, wrong login).
- Use Case Testing (real-world donation scenarios).

5.3.6 WHITE BOX TESTING

- Tested internal logic and code paths.
- Example: Checking database queries, campaign approval logic, and conditional branches.

Techniques used:

- Statement Coverage (every line of code executed).
- Branch Coverage (all true/false decisions tested).
- Condition Coverage (multiple conditions in payment and authentication tested).
- Path Coverage (all code paths checked).

5.4 MODIFICATIONS AND IMPROVEMENTS

Modifications and improvements are applied after development to enhance performance and usability.

Modifications:

- Fixing bugs in campaign approval or donation logging.
- Correcting errors in database queries.
- Updating payment gateway integration.

Improvements:

- Adding multi-factor authentication for enhanced security.
- Real-time notifications for donations received.
- Advanced reporting and analytics for fundraisers.
- Mobile-friendly design for broader accessibility.

These modifications keep the system functional and reliable, while improvements make it more advanced and user-friendly. Both are essential to maintain software quality and adapt to user needs.